

# Часть I: Введение

## 1. Что такое Affinity Scripting

Affinity Scripting — это JavaScript SDK для автоматизации работы с Affinity Designer, Photo и Publisher. Скрипты выполняются внутри приложения и имеют полный доступ к документам, слоям, фигурам, цветам, тексту и другим объектам.

### Возможности

- Создание и редактирование документов
- Работа с векторными фигурами и кривыми
- Управление цветами, заливками, градиентами
- Текстовое форматирование
- Эффекты слоёв (тени, свечение, барельеф)
- AI-команды (генерация изображений, удаление фона)
- Экспорт в различные форматы
- Создание пользовательских диалогов

### Ограничения

- **Файловая система:** доступ только к Desktop ( `app.userDesktopPath` )
- **Рендеринг:** максимальный размер JPEG — 1024px
- **Настройки:** `NOT_ALLOWED` = ограничения в настройках Affinity
- **Скрипты:** должны быть прямо выполняемыми (без `module.exports.main` )

### Версии

- Affinity Designer 2.0+
- Affinity Photo 2.0+
- Affinity Publisher 2.0+
- MCP-сервер для подключения к IDE

---

## 2. Настройка окружения

### Подключение MCP-сервера

MCP-сервер Affinity подключается через MiMo Code:

```
{
  "mcp": {
    "affinity": {
      "type": "remote",
      "url": "http://[::1]:6767/sse",
      "enabled": true
    }
  }
}
```

## Проверка подключения

```
mimo mcp list
```

Должно отобразиться:

- ✓ affinity connected

## Первые шаги

1. Прочитайте преамбулу (обязательно перед каждым скриптом)
2. Выполните простой скрипт
3. Визуализируйте результат

---

## 3. Преамбула — правила и соглашения

**ВАЖНО:** Всегда читайте преамбулу перед написанием скрипта!

### Обязательные правила

1. **Всегда** читайте преамбулу и релевантные файлы документации
2. **Всегда** ищите `search_sdk_hints` перед экспериментами
3. **Всегда** ищите имена всех MCP-инструментов перед началом
4. **Немедленно** вызывайте `add_sdk_hint` после решения проблемы

### Подключение модулей

```
// Canonical путь — всегда через `/module`  
const { Document } = require('/document');  
const { Shape } = require('/shapes');  
const { Node } = require('/nodes');  
const { Colour } = require('/colours');
```

### Вывод результатов

Скрипт **не возвращает вывод напрямую** — используйте `console.log()` :

```
console.log(`Результат: ${result}`);
```

### Создание объектов

- Если класс имеет метод `create` или `createDefault` — используйте их
- Иначе используйте `new ClassName()`

### Перечисления (Enums)

Все enum-классы имеют свойства `keys` , `values` , `entries` (НЕ методы!):

```
const { BlendMode } = require('/commands');  
console.log(BlendMode.keys);    // ['Normal', 'Darken', ...]
```

```
console.log(BlendMode.values); // [0, 1, ...]
```

## Работа со страницами

**ВАЖНО:** `doc.setCurrentSpread()` не существует! Используйте команду:

```
const { DocumentCommand } = require('/commands');

// НЕПРАВИЛЬНО:
// doc.setCurrentSpread(spread);

// ПРАВИЛЬНО:
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));
```

Устанавливайте текущую страницу только если она ещё не текущая — установка очищает выделение.

## Файловая система

Доступ только к Desktop:

```
const desktop = app.userDesktopPath;
```

---

\newpage

# Часть II: Основы

## 4. Первый скрипт

### Hello World

```
const { app } = require('/application');

console.log(`Affinity ${app.productFullName} ${app.version}`);
console.log(`Платформа: ${app.platformName}`);
console.log(`Desktop: ${app.userDesktopPath}`);
```

### Создание документа

```
const { Document, NewDocumentOptions } = require('/document');

const options = NewDocumentOptions.create();
options.width = 1920;
options.height = 1080;
options.dpi = 72;

const doc = Document.createWithSize(options);
console.log(`Создан документ: ${doc.name}`);
```

## Добавление фигуры

```
const { Shape } = require('/shapes');

const doc = app.documents.current;
const spread = doc.spreads.first;
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));

const rect = Shape.createRectangle(spread);
rect.width = 200;
rect.height = 100;

console.log(`Прямоугольник: ${rect.width}x${rect.height}`);
```

## Визуализация результата

```
// Через MCP-инструмент:
// affinity_render_spread(document_session_uuid="...", spread_index=0)
```

## Упражнение 1

Создайте документ 800x600 с тремя фигурами: прямоугольник, круг, звезда. Выведите размеры каждой фигуры в консоль.

---

# 5. Application и Document

## Application (singleton)

```
const { app } = require('/application');

// Информация о приложении
console.log(`Версия: ${app.version}`);
console.log(`Платформа: ${app.platformName}`);

// Документы
const docs = app.documents.all; // все открытые
const current = app.documents.current; // текущий
const loaded = app.documents.load(path); // загрузить

// Диалоги
app.alert('Сообщение', 'Заголовок');
const confirmed = app.confirm('Вы уверены?', 'Подтверждение');
const text = app.prompt('Введите текст:', 'Ввод', 'По умолчанию');
```

## Document

```

const doc = app.documents.current;

// Свойства
console.log(`Имя: ${doc.name}`);
console.log(`Размер: ${doc.width}x${doc.height}`);

// Страницы (Spreads)
const spreads = doc.spreads;
doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.spreads.first));

// Выделение
const selection = doc.selection;
doc.selectAll();
doc.deleteSelection();

// Сохранение
doc.save();
doc.saveAs(`${app.userDesktopPath}/output.afdesign`);

// История
doc.undo();
doc.redo();

```

## 6. Модули и require()

### Canonical путь подключения

```

// Всегда через `/module` (с косой чертой)
const { Document } = require('/document');
const { Shape } = require('/shapes');
const { Node } = require('/nodes');
const { Colour } = require('/colours');

```

### Доступные модули

| Модуль       | Назначение                              |
|--------------|-----------------------------------------|
| /application | Application, документы, UI              |
| /document    | Document, создание, сохранение, экспорт |
| /commands    | 150+ команд                             |
| /geometry    | Transform, Curve, Point, Rectangle      |
| /nodes       | Node (базовый класс слоёв)              |
| /shapes      | Shape (22 типа фигур)                   |
| /colours     | Colour, Gradient, SVG11                 |

|             |                                 |
|-------------|---------------------------------|
| /dialog     | Dialog (создание UI)            |
| /selections | Selection                       |
| /fills      | FillDescriptor, GradientFill    |
| /linestyle  | LineStyle, LineStyleDescriptor  |
| /story      | Story, StoryBuilder             |
| /collection | Collection (леннивые коллекции) |

## 7. Вывод результатов

### console.log()

```
console.log(`Текст: ${variable}`);
console.log(`Объект:`, JSON.stringify(obj));
```

### Визуализация через MCP

```
// Рендер страницы
// affinity_render_spread(document_session_uuid="...", spread_index=0)

// Рендер выделенного узла
// affinity_render_selection(document_session_uuid="...")
```

### Отладка

```
// Проверка типа узла
console.log(`Тип: ${node[Symbol.toStringTag]}`);
console.log(`Это группа: ${node.isGroupNode}`);
console.log(`Это фигура: ${node.isShapeNode}`);
console.log(`Это текст: ${node.isFrameTextNode}`);
```

\newpage

# Часть III: Объекты и геометрия

## 8. Узлы (Nodes)

### Базовый класс Node

```
const node = doc.selection.first;

// Свойства
```

```

console.log(`Имя: ${node.name}`);
console.log(`Видимый: ${node.visible}`);
console.log(`Заблокирован: ${node.locked}`);
console.log(`Прозрачность: ${node.opacity}`);

// Навигация
const parent = node.parent;
const firstChild = node.firstChild;
const lastChild = node.lastChild;
const nextSibling = node.nextSibling;
const previousSibling = node.previousSibling;

// Дочерние узлы
node.children.forEach(child => {
  console.log(`Дочерний: ${child.name}`);
});

// Рекурсивный перебор
node.children.all.forEach(descendant => {
  console.log(`Потомок: ${descendant.name}`);
});

```

## Типы узлов

```

node.isGroupNode      // группа
node.isShapeNode      // фигура
node.isFrameTextNode  // текстовый фрейм
node.isVectorNode     // векторный узел
node.isImageNode      // изображение
node.isPolyCurveNode  // составная кривая

```

## Трансформации узлов

```

node.x = 100;
node.y = 200;
node.width = 300;
node.height = 200;
node.rotation = Math.PI / 4; // 45 градусов
node.horizontalFlip = true;

```

## Упражнение 2

Напишите функцию `inspectNode(node, depth)`, которая рекурсивно обходит дерево узлов и выводит имя, тип и видимость каждого узла с отступом.

# 9. Фигуры (Shapes)

## Создание фигур

```

const { Shape } = require('/shapes');

// Прямоугольники
const rect = Shape.createRectangle(spread);
const roundRect = Shape.createRoundedRectangle(spread);

// Эллипсы
const ellipse = Shape.createEllipse(spread);

// Многоугольники
const polygon = Shape.createPolygon(spread);
const star = Shape.createStar(spread);
const triangle = Shape.createTriangle(spread);

// Специальные
const heart = Shape.createHeart(spread);
const cloud = Shape.createCloud(spread);
const spiral = Shape.createSpiral(spread);

```

## Параметры фигур

```

// Прямоугольник
rect.width = 200;
rect.height = 100;
rect.absoluteSizes = true;
rect.topLeft.setRadius(10, rect.width, rect.height);
rect.useSingleRadius = true;

// Звезда
star.points = 5;
star.innerRadius = 0.5;
star.curvedEdges = true;

// Полигон
polygon.sides = 6;
polygon.curve = 0.5;
polygon.smoothPoints = true;

// Спираль
spiral.turns = 5;
spiral.decay = 0.5;

// Шестерёнка
cog.teeth = 8;
cog.innerRadius = 0.7;
cog.holeRadius = 0.3;

```

## QR-коды

```
const { QRPayloadURL, QRPayloadEmail } = require('/shapes');

const qr = Shape.createQRCode(spread);
const payload = QRPayloadURL.create();
payload.url = 'https://example.com';
qr.payload = payload;
```

## 10. Кривые и геометрия

### Точки и прямоугольники

```
const { Point, Vector, Rectangle, Size } = require('/geometry');

const pt = new Point(100, 200);
const rect = new Rectangle(0, 0, 1920, 1080);
const size = new Size(100, 200);
```

### Трансформации

```
const { Transform } = require('/geometry');

// Создание
const translate = Transform.createTranslate(100, 50);
const rotate = Transform.createRotate(Math.PI / 4);
const scale = Transform.createScale(2, 2);

// Композиция
const xf = Transform.createIdentity()
    .translate(100, 50)
    .rotate(Math.PI / 4)
    .scale(2, 2);

// Применение к точке
const transformed = xf.applyToPoint(point);
```

### CurveBuilder

```
const { CurveBuilder } = require('/geometry');

const builder = CurveBuilder.create();
builder.beginXY(0, 0);
builder.lineToXY(100, 0);
builder.lineToXY(100, 100);
builder.close();
const curve = builder.createCurve();
```

## Curve

```
const { Curve } = require('/geometry');

const rect = new Rectangle(0, 0, 200, 100);
const rectCurve = Curve.createRectangle(rect);
const ellipseCurve = Curve.createEllipse(rect);
const line = Curve.createLineXY(0, 0, 100, 100);

// Работа с точками
console.log(`Точек: ${curve.pointCount}`);
const point = curve.getPoint(0);
curve.setPoint(0, new Point(50, 50));
```

## Arc-length sampling

```
function buildArcTable(beziers) {
  const tbl = [];
  let cum = 0;
  for (let bi = 0; bi < beziers.length; bi++) {
    const b = beziers[bi];
    let prev = evalBez(b, 0);
    if (bi === 0) tbl.push({ bi, t: 0, cum: 0 });
    for (let s = 1; s <= 200; s++) {
      const t = s / 200;
      const pt = evalBez(b, t);
      cum += dist(pt, prev);
      tbl.push({ bi, t, cum });
      prev = pt;
    }
  }
  return tbl;
}
```

---

## 11. Цвета и заливки

### Создание цветов

```
const { RGBA8, RGB8, CMYK8, HSLf, SVG11 } = require('/colours');

const red = RGBA8(255, 0, 0, 255);
const blue = RGB8(0, 0, 255);
const cyan = CMYK8(255, 0, 0, 0);
const green = HSLf(120, 1.0, 0.5);

// SVG именованные цвета
```

```
const crimson = SVG11.crimson;
const random = SVG11.random();
```

## Заливки

```
const { FillDescriptor, GradientFill } = require('/fills');
const { Gradient } = require('/colours');

// Сплошная заливка
const solidFill = FillDescriptor.createSolid(colour, BlendMode.Normal);

// Градиент
const stops = [
  { colour: RGBA8(255, 0, 0, 255), position: 0 },
  { colour: RGBA8(0, 0, 255, 255), position: 1 }
];
const gradient = Gradient.create(stops);
const gradientFill = GradientFill.create(gradient, 0);
```

## Чтение заливки

```
const fd = node.brushFillInterface.fillDescriptor;
const fill = fd.fill;
if (fill.fillType.value === FillType.Solid.value) {
  const rgba = fill.colour.rgba8;
  console.log(`R:${rgba.r} G:${rgba.g} B:${rgba.b}`);
}
```

## Упражнение 3

Создайте 5 прямоугольников с градиентной заливкой от красного к синему, с разной прозрачностью (0.2, 0.4, 0.6, 0.8, 1.0).

---

\newpage

# Часть IV: Команды и операции

## 12. Система команд

### Паттерн выполнения команд

```
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');

// Создание выделения
const sel = Selection.create(doc, node);
```

```
// Выполнение команды
const cmd = DocumentCommand.createTransform(sel, transform);
doc.executeCommand(cmd);
```

## Compound команды (пакетные операции)

```
const { CompoundCommandBuilder } = require('/commands');

const builder = CompoundCommandBuilder.create();
builder.addCommand(DocumentCommand.createSetOpacity(sel, 0.5));
builder.addCommand(DocumentCommand.createSetBlendMode(sel, BlendMode.Multiply));
doc.executeCommand(builder.createCommand());
```

## Preview режим

```
// Preview (не сохраняется)
doc.executeCommand(cmd, true);

// Применить (сохраняется)
doc.executeCommand(cmd, false);

// Очистить все preview
doc.executeCommand(DocumentCommand.createClearPreviews());
```

## Undo/Redo

```
// Сохранить позицию истории
const historyStart = doc.history.position;

// ... выполнить операции ...

// Отменить всё
doc.history.position = historyStart;
```

## Добавление узлов

```
const { AddChildNodesCommandBuilder, NodeChildType } = require('/commands');
const { ContainerNodeDefinition, ShapeNodeDefinition } = require('/nodes');

const builder = AddChildNodesCommandBuilder.create();
builder.setInsertionTarget(doc.currentSpread);

// Контейнер
builder.addContainerNode(ContainerNodeDefinition.create('Группа'));

// Фигура
const shape = ShapeRectangle.create();
```

```
const nodeDef = ShapeNodeDefinition.create(
  shape,
  new Rectangle(x, y, w, h),
  FillDescriptor.createSolid(colour)
);
builder.addShapeNode(nodeDef);

doc.executeCommand(builder.createCommand(false, NodeChildType.Main));
```

---

## 13. Выделение (Selection)

### Создание выделения

```
const { Selection } = require('/selections');

// Выделение одного узла
const sel = Selection.create(doc, node);

// Выделение нескольких узлов
const sel = Selection.create(doc, [node1, node2, node3], true);
```

### Итерация выделения

```
const sel = doc.selection;
if (sel && sel.length > 0) {
  for (let i = 0; i < sel.length; i++) {
    const node = sel.at(i).node;
    console.log(`Узел ${i}: ${node.name}`);
  }
}
```

### Поиск по цвету

```
function findNodesByColor(node, targetColor) {
  const results = [];
  if (node.hasBrushFill) {
    const fill = node.brushFillInterface.fillDescriptor.fill;
    if (fill.colour && coloursMatch(fill.colour, targetColor)) {
      results.push(node);
    }
  }
  let child = node.firstChild;
  while (child) {
    results.push(...findNodesByColor(child, targetColor));
    child = child.nextSibling;
  }
}
```

```
    return results;
}
```

## Упражнение 4

Напишите скрипт, который находит все фигуры с заливкой цвета RGBA(255, 0, 0, 255) и меняет их цвет на RGBA(0, 0, 255, 255).

## Удаление объектов

**ВАЖНО:** `spread.children` включает **все** узлы, включая артборды! Не удаляйте их все.

```
// НЕПРАВИЛЬНО – удалит артборды тоже:
const nodes = [];
children.forEach(c => nodes.push(c)); // включает артборды!

// ПРАВИЛЬНО – фильтруем по типу:
const nodes = [];
children.forEach(c => {
  if (c.isShapeNode || c.isGroupNode || c.isFrameTextNode ||
      c.isVectorNode || c.isImageNode || c.isPolyCurveNode) {
    nodes.push(c);
  }
});

// Или удаляем только внутри артбордов:
doc.spreads.forEach(spread => {
  spread.children.forEach(artboard => {
    const objects = [];
    artboard.children.forEach(obj => objects.push(obj));
    if (objects.length > 0) {
      const sel = Selection.create(doc, objects, true);
      doc.executeCommand(DocumentCommand.createDeleteSelection(sel));
    }
  });
});
```

# 14. Трансформации

## Базовые трансформации

```
const { Transform } = require('/geometry');
const { Selection } = require('/selections');

const sel = Selection.create(doc, node);

// Перемещение
doc.executeCommand(DocumentCommand.createTransform(
  sel, Transform.createTranslate(100, 50)
));
```

```

// Поворот вокруг центра
const cx = node.x + node.width / 2;
const cy = node.y + node.height / 2;
const rotateXf = Transform.createTranslate(cx, cy)
    .multiply(Transform.createRotate(Math.PI / 4))
    .multiply(Transform.createTranslate(-cx, -cy));
doc.executeCommand(DocumentCommand.createTransform(sel, rotateXf));

// Масштабирование
doc.executeCommand(DocumentCommand.createTransform(
    sel, Transform.createScale(2, 2)
));

```

## Групповая трансформация

```
doc.groupTransform(nodes, transform);
```

## 15. Эффекты слоёв

```

// Тень
doc.addLayerEffect(node, LayerEffectType.InnerShadow, {
    colour: RGBA8(0, 0, 0, 128),
    offset: 5,
    blur: 10,
    opacity: 0.5
});

// Свечение
doc.addLayerEffect(node, LayerEffectType.OuterGlow, {
    colour: RGBA8(255, 255, 0, 255),
    blur: 20,
    opacity: 0.8
});

// Барельеф
doc.addLayerEffect(node, LayerEffectType.BevelEmboss, {
    style: BevelEmbossStyle.Inner,
    depth: 100,
    size: 5
});

// Управление эффектами
const effects = node.layerEffects;
effects.innerShadow.enabled = true;
effects.removeEffect(LayerEffectType.InnerShadow);

```

## Часть V: Текст и UI

### 16. Текст

#### StoryBuilder

```
const { StoryBuilder } = require('/story');

const builder = new StoryBuilder();
builder.appendText('Жирный текст', { bold: true });
builder.appendText(' и ');
builder.appendText('курсив', { italic: true });

textFrame.setStory(builder.build());
```

#### Атрибуты текста

```
const { GlyphAtts, ParagraphAtts } = require('/glyphatts');

const glyphAtts = new GlyphAtts();
glyphAtts.bold = true;
glyphAtts.fontSize = 24;
glyphAtts.fontFamily = 'Arial';
glyphAtts.colour = RGBA8(255, 0, 0, 255);

const paraAtts = new ParagraphAtts();
paraAtts.alignment = ParagraphAlignment.Center;
```

---

### 17. Диалоги

#### Базовый диалог

```
const { Dialog, DialogResult } = require('/dialog');

const dlg = Dialog.create('Название');
dlg.initialWidth = 360;

const col = dlg.addColumn();
const grp = col.addGroup('Параметры');

// Редактор значений
const slider = grp.addUnitValueEditor('Параметр', UnitType.Pixel, UnitType.Pixel, 10, 1, 100);
slider.precision = 0;
slider.showPopupSlider = true;

// Переключатель
```

```

const switchCtrl = grp.addSwitch('Опция', false);

// Комбобокс
const combo = grp.addComboBox('Режим', ['Вариант 1', 'Вариант 2'], 0);

// Чекбокс
const check = grp.addCheckBox('Флажок', true);

// Кнопки действий
const actGrp = col.addGroup('');
const btns = actGrp.addButtonSet('', ['Preview', 'Apply'], 0);
btns.isFullWidth = true;

// Статусный текст
const status = actGrp.addStaticText('', 'Готово');
status.isFullWidth = true;

const result = dlg.runModal();

```

## Паттерн Preview → Apply

```

let previewActive = false;
let cmdCount = 0;

function applyPreview() {
  if (previewActive) {
    for (let i = 0; i < cmdCount; i++) {
      doc.executeCommand(DocumentCommand.createUndo());
    }
    previewActive = false;
  }
  const cmd = createMyCommand(params);
  doc.executeCommand(cmd, true);
  previewActive = true;
  cmdCount = 1;
}

while (running) {
  const result = dlg.runModal();
  if (result.value !== DialogResult.Ok.value) {
    if (previewActive) {
      for (let i = 0; i < cmdCount; i++) {
        doc.executeCommand(DocumentCommand.createUndo());
      }
    }
    running = false;
  } else if (btns.selectedIndex === 0) {
    applyPreview();
  } else {
    applyPreview();
    running = false;
  }
}

```

```
}  
}
```

---

## 18. Экспорт

### Базовый экспорт

```
const { FileExportOptions, FileExportArea } = require('/document');  
  
const options = FileExportOptions.create();  
options.area = FileExportArea.CurrentSpread;  
  
doc.export(`${app.userDesktopPath}/output.png`, options);
```

### Экспорт с ограничением размера

```
function exportWithSizeLimit(doc, path, maxSizeKB) {  
  let quality = 0.92;  
  while (quality > 0.1) {  
    const options = FileExportOptions.create();  
    options.quality = quality;  
    doc.export(path, options);  
    const size = getFileSize(path);  
    if (size <= maxSizeKB * 1024) break;  
    quality -= 0.05;  
  }  
}
```

---

\newpage

## Часть VI: Продвинутые темы

### 19. AI-команды

```
// Генерация изображения  
const imageNode = doc.generateImage('Красный закат над морем', {  
  width: 1024, height: 768  
});  
  
// Генеративное редактирование  
doc.generativeEditImage(imageNode, 'Добавить птиц на небе');  
  
// Удаление фона  
doc.removeBackground(imageNode);  
  
// Выделение объекта
```

```
doc.selectSubject(imageNode);

// Раскрашивание
doc.colourise(imageNode);
```

---

## 20. Файловая система

```
const { File, Directory } = require('fs.js');

const desktop = app.userDesktopPath;

// Чтение
const file = File.open(`${desktop}/data.txt`, 'r');
const content = file.readAll();
file.close();

// Запись
const outFile = File.open(`${desktop}/output.txt`, 'w');
outFile.write('Hello, World!');
outFile.close();
```

---

## 21. Сеть

```
const { HttpRequest, RequestMethod } = require('network.js');

const request = HttpRequest.create('https://api.example.com/data', RequestMethod.GET);
request.setTimeoutInSec(30);
request.setHeaderValue('Authorization', 'Bearer token123');

const { response } = request.do();
console.log(`Status: ${response.statusCode}`);
console.log(`Body: ${response.content}`);
```

---

## 22. Таймеры и буферы

### Таймеры

```
setTimeout(1000, (errorCode) => {
  console.log('Прошло 1 секунду');
});

setInterval(500, (errorCode) => {
  console.log('Каждые 500мс');
});
```

## Буферы

```
const { Buffer } = require('/buffer');

const buf = Buffer.create(1024);
const str8 = Buffer.utf8('Hello');
const string = buf.toString('utf8');
```

## 23. Коллекции

```
const { Collection } = require('/collection');

const range = Collection.range(0, 10);
const filtered = range.filter(x => x > 5);
const mapped = range.map(x => x * 2);
const sum = range.reduce((acc, x) => acc + x, 0);
```

\newpage

# Часть VII: Отладка и лучшие практики

## 24. Отладка скриптов

### console.log для отладки

```
// Вывод информации об узле
console.log(`Узел: ${node.name}`);
console.log(`Тип: ${node[Symbol.toStringTag]}`);
console.log(`Позиция: (${node.x}, ${node.y})`);
console.log(`Размер: ${node.width}x${node.height}`);

// Вывод дерева узлов
function inspectNode(node, depth = 0) {
  const indent = ' '.repeat(depth);
  console.log(`${indent}${node.name} [${node[Symbol.toStringTag]}]`);
  let child = node.firstChild;
  while (child) {
    inspectNode(child, depth + 1);
    child = child.nextSibling;
  }
}
```

### Визуальная отладка

```
// Используйте render для проверки результатов:
// affinity_render_spread(document_session_uuid="...", spread_index=0)
// affinity_render_selection(document_session_uuid="...")
```

### Частые ошибки

| Ошибка        | Причина                           | Решение                         |
|---------------|-----------------------------------|---------------------------------|
| NOT_ALLOWED   | Ограничения в настройках Affinity | Включите доступ к AI/FS/Network |
| Пустой render | Неправильный UUID сессии          | Проверьте document_session_uuid |
| undefined     | Модуль не подключён               | Добавьте require('/module')     |
| Неверный цвет | Использованы неверные диапазоны   | RGBA: 0-255, HSL: 0-1           |

## 25. Обработка ошибок

```
try {
  const result = riskyOperation();
  console.log(`Успех: ${result}`);
} catch (error) {
  console.error(`Ошибка: ${error.message}`);
  app.alert(`Ошибка: ${error.message}`, 'Ошибка');
}
```

## 26. Производительность

### Батч-команды

```
// Вместо множества отдельных команд
const builder = CompoundCommandBuilder.create();
for (const node of nodes) {
  builder.addCommand(DocumentCommand.createTransform(sel, xf));
}
doc.executeCommand(builder.createCommand());
```

### Undo батчем

```
const historyStart = doc.history.position;
// ... все операции ...
// Одна отмена отменяет всё
doc.history.position = historyStart;
```

## 27. Паттерны проектирования

## Preview/Apply паттерн

```
// 1. Показать preview
doc.executeCommand(cmd, true);
// 2. Пользователь настраивает
// 3. Применить или отменить
doc.executeCommand(cmd, false);
```

## Сохранение настроек в тегах

```
const { TagInterfaceApi } = require('affinity:dom');

function saveSettings(doc, key, value) {
  const node = doc.layers.first;
  const ti = TagInterfaceApi.fromNode(node.handle);
  doc.executeCommand(
    DocumentCommand.createSetTagValueForKey(
      node.selfSelection, key, JSON.stringify(value)
    )
  );
}
```

\newpage

# Часть VIII: Практикум

## 28. Рецепт 1: Создание сетки артбордов

```
const doc = app.documents.current;
const cols = 3, rows = 2;
const width = 400, height = 300, gap = 20;

for (let row = 0; row < rows; row++) {
  for (let col = 0; col < cols; col++) {
    doc.addArtboard({
      x: col * (width + gap),
      y: row * (height + gap),
      width: width,
      height: height,
      name: `Artboard ${row * cols + col + 1}`
    });
  }
}
```

## 29. Рецепт 2: Автоматизация цвета

```

const doc = app.documents.current;
doc.spreads.forEach(spread => {
  spread.descendants.forEach(node => {
    if (node.fill && node.fill.colour) {
      const oldColour = node.fill.colour;
      if (oldColour.r === 255 && oldColour.g === 0 && oldColour.b === 0) {
        node.fill.colour = RGBA8(0, 0, 255, 255);
      }
    }
  });
});

```

## 30. Рецепт 3: Экспорт карусели

```

const doc = app.documents.current;
const desktop = app.userDesktopPath;

doc.spreads.forEach((spread, index) => {
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));
  const options = FileExportOptions.create();
  options.area = FileExportArea.CurrentSpread;
  doc.export(`${desktop}/page_${index + 1}.png`, options);
});

```

## 31. Рецепт 4: Генеративный паттерн

### Центрирование объекта на артборде

**ВАЖНО:** `getSpreadExtents()` возвращает координаты в SDK-единицах. Для центрирования:

```

const { app } = require('/application');
const { DocumentCommand } = require('/commands');
const { ShapeRectangle } = require('/shapes');
const { RGBA8 } = require('/colours');
const { FillDescriptor } = require('/fills');
const { BlendMode } = require('affinity:common');
const { AddChildNodesCommandBuilder, NodeChildType } = require('/commands');
const { ShapeNodeDefinition } = require('/nodes');
const { Rectangle } = require('/geometry');

const doc = app.documents.current;
const spread = doc.spreads.first;
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));

// Получаем размер артборда
const extents = spread.getSpreadExtents();
const cx = extents.x + extents.width / 2;
const cy = extents.y + extents.height / 2;

```

```
// Создаём квадрат 400x400 в центре
const size = 400;
const r = ShapeRectangle.create();
r.setAbsoluteSizes(true, size, size);

const def = ShapeNodeDefinition.create(
  r,
  new Rectangle(cx - size/2, cy - size/2, size, size),
  FillDescriptor.createSolid(rgba(0, 0, 255, 255), BlendMode.Normal)
);

const b = AddChildNodesCommandBuilder.create();
b.setInsertionTarget(spread);
b.addShapeNode(def);
doc.executeCommand(b.createCommand(false, NodeChildType.Main));

console.log(`Blue square at center: (${cx}, ${cy})`);
```

## Scatter — рассеивание объектов

```
function scatterObjects(nodes, count, area) { const results = []; for (let i = 0; i < count; i++) { const node = nodes[i % nodes.length]; const clone = node.duplicate(); clone.x = area.x + Math.random() * area.width; clone.y = area.y + Math.random() * area.height; clone.rotation = Math.random() * Math.PI * 2; results.push(clone); } return results; }
```

---

## 32. Рецепт 5: Текстовый макет

```
```js
const { StoryBuilder } = require('/story');

const textFrame = doc.addFrameText();
textFrame.x = 50;
textFrame.y = 50;
textFrame.width = 400;
textFrame.height = 200;

const builder = new StoryBuilder();
builder.appendText('Заголовок\n', { bold: true, fontSize: 24 });
builder.appendText('Основной текст документа.', { fontSize: 14 });

textFrame.setStory(builder.build());
```

\newpage

## Приложения

## Приложение А: МСР-инструменты

| #  | Инструмент                            | Назначение                      |
|----|---------------------------------------|---------------------------------|
| 1  | affinity_execute_script               | Выполнение JavaScript           |
| 2  | affinity_read_sdk_documentation_topic | Чтение документации SDK         |
| 3  | affinity_list_sdk_documentation       | Список тем документации         |
| 4  | affinity_list_library_scripts         | Список библиотечных скриптов    |
| 5  | affinity_read_library_script          | Чтение скрипта из библиотеки    |
| 6  | affinity_save_script_to_library       | Сохранение скрипта в библиотеку |
| 7  | affinity_render_selection             | Рендер выделенного узла         |
| 8  | affinity_render_spread                | Рендер страницы                 |
| 9  | affinity_search_sdk_hints             | Поиск подсказок SDK             |
| 10 | affinity_add_sdk_hint                 | Добавление подсказки            |
| 11 | affinity_report_sdk_issue             | Сообщение об ошибке SDK         |

## Приложение В: Модули SDK

| require()    | Модуль      | Ключевые экспорты                                  |
|--------------|-------------|----------------------------------------------------|
| /application | Application | app, BuildKind, UiParadigm                         |
| /document    | Document    | Document, NewDocumentOptions, FileExportOptions    |
| /commands    | Commands    | DocumentCommand, CompoundCommandBuilder            |
| /geometry    | Geometry    | Transform, Curve, CurveBuilder, Point              |
| /nodes       | Nodes       | Node, ShapeNodeDefinition, ContainerNodeDefinition |
| /shapes      | Shapes      | Shape, ShapeRectangle, ShapeStar, QRPayloadURL     |
| /colours     | Colours     | Colour, RGBA8, Gradient, SVG11                     |
| /dialog      | Dialog      | Dialog, DialogResult                               |
| /selections  | Selections  | Selection                                          |
| /fills       | Fills       | FillDescriptor, GradientFill                       |
| /linestyle   | LineStyle   | LineStyle, LineStyleDescriptor                     |
| /story       | Story       | Story, StoryBuilder                                |
| /collection  | Collection  | Collection, SpanCollection                         |

# Приложение С: Типы фигур

| Тип       | Класс          | Параметры                                    |
|-----------|----------------|----------------------------------------------|
| Rectangle | ShapeRectangle | width, height, cornerRadius, useSingleRadius |
| Ellipse   | ShapeEllipse   | —                                            |
| Star      | ShapeStar      | points, innerRadius, curvedEdges             |
| Polygon   | ShapePolygon   | sides, curve, smoothPoints                   |
| Triangle  | ShapeTriangle  | position                                     |
| Diamond   | ShapeDiamond   | position                                     |
| Heart     | ShapeHeart     | spread                                       |
| Cloud     | ShapeCloud     | bubbles, innerRadius                         |
| Spiral    | ShapeSpiral    | turns, decay, style                          |
| Cog       | ShapeCog       | teeth, innerRadius, holeRadius               |
| Arrow     | ShapeArrow     | thickness, leftEnd, rightEnd                 |
| QRCode    | ShapeQRCode    | payload (URL, Email, Phone, WiFi...)         |

# Приложение D: Community-скрипты

70+ скриптов из [JiriKrblich/Affinity-Community-Scripts](#).

Ключевые скрипты для изучения:

| Скрипт              | Что изучить                                   |
|---------------------|-----------------------------------------------|
| Blend Tool          | Интерполяция кривых, arc-length sampling      |
| Bento Box Generator | Генеративные алгоритмы, рекурсивное разбиение |
| Randomize Objects   | Шумовые функции (Perlin, Gaussian)            |
| Zig Zag Effect      | Модификация кривых, preview/apply паттерн     |
| Distribute on Paths | Размещение вдоль путей, нормали               |

Все скрипты доступны в папке `community-scripts/` .

Учебник создан на основе полной документации Affinity MCP-сервера и 70+ community-скриптов.